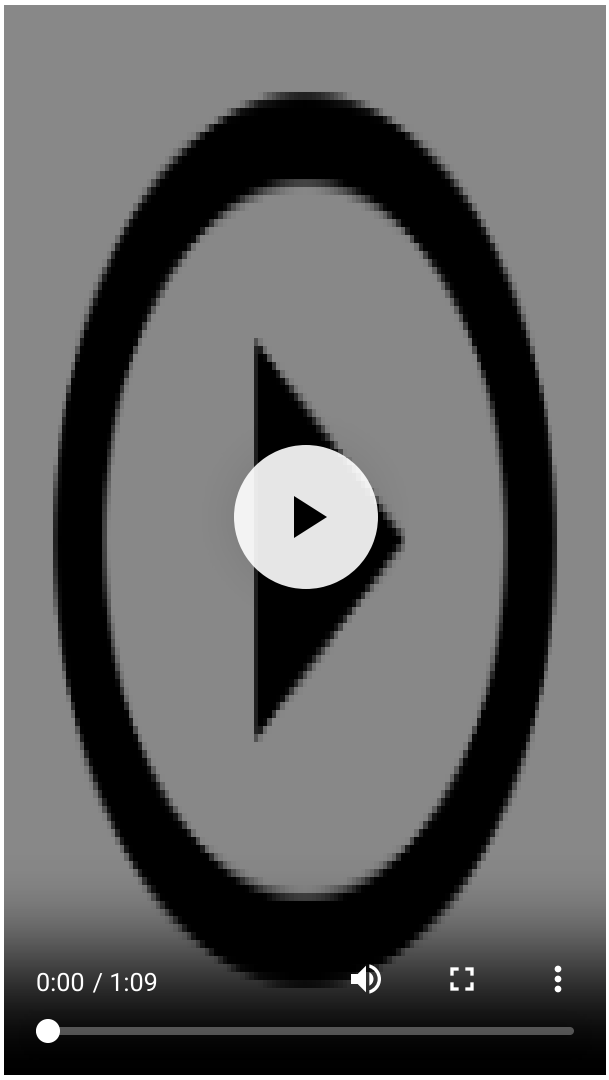
Activity 4.4: Integrate the Mobile and Web Applications



Remember, your goal is to demonstrate one integrated prototype that addresses the approved stakeholder requirements.

Supporting Video: Connecting an Application to Supabase

Watch the following Supabase tutorial:

Getting Started with Flutter and Supabase (https://www.youtube.com/watch?v=F2j6Q-4nLEE) .

The video demonstrates how an application connects to Supabase and uses database operations. It focuses on Flutter rather than a combined Flutter and ASP.NET system. Apply the same shared-backend principle to both of your applications.

You can also refer to the official Supabase Flutter Quickstart (https://supabase.com/docs/guides/getting-started/quickstarts/flutter) and Supabase C# (https://supabase.com/docs/reference/csharp/installing) Documentation for the respective application connections.

While Watching

Consider these questions:

1. How does an application connect to Supabase?
2. How is information saved to the database?
3. How does the application retrieve information?
4. What must your group do to ensure that Flutter and ASP.NET work with the same data?

Activity at a Glance

You will complete	**Main outcome**
Check both application connections	One shared Supabase backend
Implement shared processes	Data moves correctly between the applications
Check user roles and access	Users can perform their approved functions
Check data consistency	Both applications use the same records and status values
Identify and correct integration problems	Improved working prototype
Update project records	Evidence of development and integration progress

What You Must Do

Step 1: Review Your Approved Integration Design

Open your updated SRS and review the system architecture, use case diagram and sequence diagrams from Unit 3.

Identify the processes that require information to move between Flutter, Supabase and ASP.NET.

Complete the following table for your project.

Shared process	**Action in Flutter**	**Action in ASP.NET**	**Shared information**

Short example: Salon appointment system

Shared process	**Action in Flutter**	**Action in ASP.NET**	**Shared information**
Appointment request	Customer submits a request	Staff view the request	Appointment details
Appointment approval	Customer views the status	Staff approve the request	Appointment status
Appointment rejection	Customer views the status	Staff reject the request	Appointment status

Your group's processes must reflect your own approved requirements.

Step 2: Confirm That Both Applications Use the Same Backend

Check the configuration of your Flutter and ASP.NET projects.

Both applications must connect to the same Supabase project and the relevant shared database tables.

Item to check	**Flutter**	**ASP.NET**
Correct Supabase project		
Required database tables		
Appropriate authentication		
Required database operations		
Approved user permissions		

Important: Using the same backend does not mean every user has the same permissions. Access must still follow the roles and requirements in your SRS.

Step 3: Check That Both Applications Use Consistent Information

Compare the database fields and values used in the two applications.

For example, if the Appointment table contains a Status field, the applications must use the same agreed status values.

Field	Flutter	ASP.NET
AppointmentID	Identifies the appointment	Identifies the same appointment
CustomerID	Associates the request with the customer	Identifies the relevant customer
Status	Displays the current status	Updates the current status

Avoid situations where one application uses Approved while the other expects a different value that it cannot interpret.

Both applications must also handle dates, required fields and record identifiers consistently.

Step 4: Demonstrate a Complete Shared Process

Select at least two important processes from your approved requirements that involve both applications. Where possible, demonstrate information flowing in both directions.

For each process, check what happens from the first user action to the final result.

Short example: Salon appointment system

1. A customer submits an appointment request in Flutter.
2. Flutter saves the request in Supabase.
3. A staff member opens the ASP.NET web application.
4. The web application retrieves the same request from Supabase.
5. The staff member approves the request.
6. ASP.NET updates the appointment status in Supabase.
7. The customer refreshes or reopens the relevant Flutter screen and sees the updated status.

The final step does not need to happen automatically unless real-time updates form part of your approved requirements. Supabase supports real-time change subscriptions, but these must be configured and implemented if needed.

Flutter stream (https://supabase.com/docs/reference/dart/stream)

Listening to Postgres Changes with Flutter (https://supabase.com/docs/guides/realtime/realtime-listening-flutter)

Step 5: Check Authentication and Access Across the System

Check the approved user roles using suitable test accounts.

User role	Expected action	Result
Mobile user	Access their permitted functions and information	
Web user	Access their permitted functions and information	

Unauthorised user	Cannot access restricted records or functions	

For example, a customer should not be able to approve another customer's appointment simply by changing information in the application.

Check that the database permissions support the intended access rules, rather than relying only on hiding buttons or pages.

Flutter installing (https://supabase.com/docs/reference/dart/installing)

C# installing (https://supabase.com/docs/reference/csharp/installing)

Step 6: Record and Correct Integration Problems

As you check the shared processes, record any problems.

Integration problem	**Possible cause**	**Action taken**
New request does not appear in ASP.NET	Incorrect query or connection	
Updated status is not shown in Flutter	Information was not retrieved again	
A user cannot retrieve an authorised record	Incorrect access policy or query	
Information is displayed incorrectly	Different field names or data formats	

Assign each problem to a group member, correct it and check the affected process again.

Keep the problem and correction records in your Project Progress Tracker.

Step 7: Demonstrate the Integrated Prototype

Prepare a short demonstration showing that your mobile and web applications work together.

Your demonstration should show:

1. The Flutter mobile application running.
2. The ASP.NET web application running.
3. A user completing an important function in one application.
4. The relevant information being saved or updated in Supabase.
5. The other application retrieving or updating that information.
6. The final result appearing correctly in the first application, where applicable.

Use suitable dummy data during the demonstration.

Step 8: Update Your Project Documentation and Tools

Document or tool	What to update
SRS	Record approved changes made during implementation
GitHub	Commit final integration changes and relevant documentation
Microsoft Project	Update completed tasks and current progress
Project Progress Tracker	Record integration work, contributions, problems and corrections

Keep the source code for both applications in the existing group repository.

What to Keep as Evidence

Evidence	What it should show
Flutter and ASP.NET screenshots	Both applications running
Screenshots or short recording of shared processes	Information passing through the shared backend
Supabase evidence	Records created or updated during the demonstration
Integration problem records	Problems identified and actions taken
GitHub commits	Code changes and group contributions
Updated Microsoft Project	Development and integration progress
Updated Project Progress Tracker	Completed work and challenges

Do not expose passwords, secret keys or confidential personal information in your evidence.

Activity Outcome

A working integrated prototype in which the Flutter mobile application and ASP.NET web application use the same Supabase backend.

The group should be able to demonstrate the approved shared processes and explain how information moves through the system.

Before Moving On

Confirm that your group has:

- connected both applications to the same Supabase project;
- implemented the shared processes identified in the SRS;
- checked that information is stored and retrieved consistently;
- confirmed that user access follows the approved requirements;
- demonstrated at least two important shared processes;
- identified and corrected integration problems;
- retained evidence of the integrated prototype;
- committed relevant changes to GitHub; and
- updated Microsoft Project and the Project Progress Tracker.

Next: Unit 4 Group Progress Checkpoint.